JS Cheat sheet

Basics

Script Placement

```
<!-- JS in a separate file -->
<script src="filename.js"></script>
<script>
    // Write JavaScript directly inside HTML
</script>
```

Output & Debugging

```
console.log(a); // Print 'a' to the browser's console
alert("Hello!"); // Show a popup message box
confirm("Are you sure?"); // OK/Cancel box; returns true (OK) or false
(Cancel)
prompt("Age?", "18"); // Input box; 2nd value "18" is the default answer
```

Conditionals

Operator	Meaning	Example
==	Equal Value	5 == "5" is true
===	Strict Equal (Value + Type)	5 === "5" is false
&&	AND (Both must be true)	(5 > 1 && 10 > 1)
!	NOT (Opposite)	!true is false

Variables & Data Types

```
let x = 10; // Standard variable; can be updated later
const y = 5; // Constant; value cannot be changed after assignment

let str = "Hello"; // String: Text inside quotes
let num = 3.14; // Number: Integers or decimals
let bool = true; // Boolean: Either true or false
let arr = [1, 2, 3]; // Array: A list of items
let obj = { id: 1, name: "luke" }; // Object: Key-value pairs
```

DOM Manipulation (Editing the Page)

Selecting Elements

```
const el = document.getElementById("myId"); // Find element by its ID
const box = document.querySelector(".myClass"); // Find first element by
CSS selector
const list = document.querySelectorAll("p"); // Find all <p> tags; returns
a list
const lastP = document.querySelector("div p:last-child"); // Find last <p>
inside a div
```

Editing Elements

```
el.innerHTML = "<b>New Text</b>"; // Change content and allow HTML tags
el.textContent = "Plain Text"; // Change text only (safer than innerHTML)
el.style.color = "blue"; // Change specific CSS style
el.classList.add("active"); // Add a CSS class name to the element
```

Conditionals & Logic

If - Else

```
if (age >= 18 && member == true) {
    status = "Enter"; // Runs if both conditions are true (AND)
} else if (age >= 18 || guest == true) {
    status = "Check ID"; // Runs if at least one is true (OR)
} else {
    status = "Blocked"; // Runs if all above are false
}
```

Switch Statement

```
switch (
    new Date().getDay() // Check the value of current day (0-6)
) {
    case 6: // If the value is 6
        text = "Saturday";
        break; // Stop and exit the switch
    case 0: // If the value is 0
        text = "Sunday";
        break;
    default: // Runs if no cases match
        text = "Weekday";
}
```

Loops

Standard For Loop

```
for (let i = 0; i < 10; i++) {
    // (Start; Condition; Step)
    console.log(i); // Runs 10 times (counts 0 to 9)
}
```

(For...of)

```
for (let item of items) {
    // Goes through every item in a list
    console.log(item); // 'item' represents the current value
}
```

While Loop

```
let energy = 3;
// Check if condition is true
while (energy > 0) {
    console.log("Working...");
    energy--; //Change value so loop eventually ends
}
```

Strings

```
let s = "JavaScript";
s.length; // Get number of characters (10)
s.toUpperCase(); // Change to "JAVASCRIPT"
s.slice(0, 4); // Get first 4 letters: "Java"
s.replace("Java", "Type"); // Swap text: result "TypeScript"
s.includes("Script"); // Check if text exists inside: true
`Value is: ${x}`; // Backticks let you put variables inside ${ }
```

Arrays

```
let fruits = ["Apple", "Banana"];

fruits.push("Orange"); // Add "Orange" to the very end
fruits.pop(); // Remove the last item in the list
```

```
fruits.shift(); // Remove the first item in the list
fruits.unshift("Kiwi"); // Add "Kiwi" to the very start

fruits.indexOf("Banana"); // Find position of item (1)
fruits.join(", "); // Turn array into string: "Kiwi, Banana"
fruitsAllCaps = fruits.map((fruit) => fruit.toUpperCase());
longFruits = fruits.filter((fruit) => fruit.length > 5); // ["Banana"]
```

Functions

```
//Arrow Function
const add = (a, b) => {
    // a and b are inputs
    return a + b; // Output the result
};

// Standard Function
function greet(name) {
    // name is the input
    return "Hi " + name; // Output a greeting
}
```

Events

```
const btn = document.querySelector("#btn");

btn.addEventListener("click", (event) => {
    // This code runs when the button is clicked
    console.log("Clicked!");
    console.log(event.target); // The exact HTML element that was clicked
});
```

Math

```
Math.round(4.4); // Round to nearest whole number (4)
Math.ceil(4.1); // Always round UP (5)
Math.floor(4.9); // Always round DOWN (4)
Math.random(); // Generate random decimal between 0 and 1
Math.max(1, 5, 10); // Pick the largest number (10)
// Get random whole number between 1 and 10:
Math.floor(Math.random() * 10) + 1;
```

Errors

```
try {
    runFunction(); // Try to run this code
} catch (err) {
    console.log(err.message); // If code fails, run this and show the error
}
```

Templates

```
const template = `<h1>Hello ${name}</h1>`;
const htmlTable = `<table>
<tr>
    <th>Name</th>
    <th>Age</th>
</tr>
${people
    .map((person) => `<tr><td>${person.name}</td><td>${person.age}</td>
</tr>`)
    .join("")}
</table>`;
```

Fetch, JSON, OOP

```
const response = await fetch("resource.json"); // get remote data
const data = await response.json(); //Parse JSON data
const str = JSON.stringify(data); //Convert data to a string:
const obj = JSON.parse(str); //Convert string back to JSON:
```

JS Classes

```
class Car {
    constructor(make, model) {//make a class (Blueprint)
        this.make = make;
        this.model = model;
    }
    drive() {//method that any instance of Car can use
        console.log(`${this.make} ${this.model} is driving!`);
    }
}
const myCar = new Car("Ford", "Mustang"); //makes a car from the blueprint
myCar.drive(); //calls the method
```